Topic: Terraform Cloud (HCP Terraform): Organization, Workspace, VCS-Driven Runs, Variables, Notifications & Cost Estimation

(Your notes, cleaned up from dictation + screenshots. Lecture 11, the last lecture of the Terraform course)

Quick recall

> **Definition (reference): Terraform Cloud** is a **managed service by HashiCorp** that automates and standardizes **Infrastructure as Code (IaC)**. It acts as a **centralized, collaborative platform for teams to build, change, and manage cloud resources** (like AWS, Azure, or GCP) **safely**.

In one line: till now we ran Terraform on our own machine (the CLI). Terraform Cloud runs the **same Terraform** for us **on HashiCorp's cloud, as SaaS**. The state, the runs, the variables, the approvals and the history all live in one place that the whole team shares.

> **Name check:** HashiCorp renamed *Terraform Cloud* to **HCP Terraform** (HCP = HashiCorp Cloud Platform) in 2024. That's why the UI says "HCP Terraform" everywhere. It's the same product, and interviewers use both names.

Different ways to use Terraform

Way	What it is	Where the run happens
Terraform CLI (Terraform Core / Community Edition)	The command-line binary we installed. It's what chapters 1 to 10 used	Your laptop / your Linux server
HCP Terraform (Terraform Cloud)	HashiCorp-hosted **SaaS**. This chapter	HashiCorp's cloud (remote execution)
Terraform Enterprise	The same platform, **self-hosted** in your own data centre or cloud (for regulated companies)	Your own infrastructure

Key capabilities (why companies use it)

Capability	What it gives you	Chapter it replaces / improves
Remote State Management	State stored in the cloud with **automatic versioning and state locking**, so there are no conflicts when many team members make changes	Ch 08: no S3 bucket + DynamoDB table to build yourself
Collaboration & VCS Integration	Connects to **GitHub, GitLab, Bitbucket**, and **automatically runs a plan** when code is pushed or a pull request is created	Running `plan` by hand
Automated & Consistent Runs	Every plan/apply runs in a **secure, isolated cloud environment**, not on engineers' laptops	"Works on my machine", differing TF versions
Governance & Policy-as-Code	**Sentinel** (and OPA) policies enforce compliance, and **cost estimation** shows the cost *before* resources are built	Manual review only
Role-Based Access Control (RBAC)	Restrict who can do what, so **only authorized users can approve changes to production**	Everyone with AWS keys can apply

Policy as Code = Sentinel *(the benefit to remember)*

One of the very good benefits of Terraform Cloud is **policy as code**, known as **Sentinel**. You write different policies and Terraform Cloud checks every plan against them **before** apply:

- **Security policies:** e.g. no security group open to `0.0.0.0/0` on port 22, S3 must be encrypted
- **Cost policies:** e.g. block the run if the estimated monthly increase is over $500
- **Configuration policies:** e.g. only `t2` / `t3` instance types, mandatory `Owner` / `CostCenter` tags

So it helps you **organize your Terraform nicely, with proper compliance rules**. The run page even has a **"Download Sentinel mocks"** button (shot 16), which gives you test data for writing and testing your own Sentinel policies.

> Sentinel and policy enforcement are a **paid-tier** feature. On the free tier you see the button but you can't enforce policies.

---

1. The hierarchy: Organization → Project → Workspace

```
Organization  (helloyello123321456)     ← your company / team. Foundation of the whole setup
 ├── Settings: Teams, Users, Variable sets, Cost estimation, Plan & billing …
 └── Project  (Default Project)          ← folder that groups workspaces (e.g. per departmen
      └── Workspace  (demorepo)          ← one per client / environment / component
           ├── Configuration  (from GitHub repo srm71118/demorepo)
           ├── Variables      (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY)
           ├── State          (versioned, one version per run)
           └── Runs           (history of every plan/apply)
```

Level	Think of it as	Notes
Organization	Your **company / team / environment**	The first thing you create after signup. Billing, users, teams and org-wide settings live here
Project	A **folder** of workspaces	Every org gets a `Default Project`. Permissions can be granted per project
Workspace	**One Terraform working directory**: code + variables + state + run history	Make separate workspaces for **different clients or environments** (dev / qa / prod), as per your requirement

> **Interview trap: HCP workspace ≠ CLI workspace.** In Ch 03, `terraform workspace new dev` gave *multiple state files for the same code in the same directory*. An **HCP Terraform workspace** is much bigger: its own code source, variables, state, permissions and run history. The usual pattern is one HCP workspace per environment (`app-dev` , `app-prod`).

---

2. Lab: from signup to a running EC2

Step 1: Create the organization (shots 03–04)

After signing up at `app.terraform.io` you land on **Organizations** → **"Create your first organization"**.

- **Business** vs **Personal**: Business is for a company or institution (handles taxes, ties the org to the business rather than to a person). We picked **Personal** for the lab.
- **Terraform organization name**: must be **unique** (globally). Allowed characters are ASCII letters, numbers, `-` and `_`. Lab name: `helloyello123321456`.

- **Email address**: used for future notifications such as **billing alerts**, and for the org avatar (via gravatar.com).

Step 2: Create a workspace and choose the workflow (shot 05)

Left menu once inside the org: **Projects · Stacks (New) · Workspaces · Registry · Usage · Settings · Explorer · HashiCorp Cloud Platform**.

Workspaces → New Workspace → "Choose your workflow". There are 3 ways:

Workflow	How runs get triggered	Best for
Version Control Workflow	**Commits / pull requests** to a connected repo	Those who need **traceability and transparency**. **This is what we used today**
CLI-Driven Workflow	`terraform plan` / `apply` on your terminal; the run streams from HCP	Those comfortable with the Terraform CLI
API-Driven Workflow	The **HCP Terraform API** (upload config + create run)	Custom integrations and pipelines (Jenkins, GitHub Actions …)

Step 3: Connect to VCS and pick the repository (shots 06–07)

The wizard has 5 steps: **Choose Type → Connect to VCS → Choose a repository → Configure settings → Variable editor**.

- **Connect to VCS**: provide your version control system (GitHub, GitLab, Bitbucket, Azure DevOps …). We chose **GitHub → GitHub App** (GitHub App Integration). This installs the HCP Terraform app on your GitHub account and you authorise which repos it can see.
- **Choose a repository**: GitHub organization `srm71118`, repository `demorepo`. HCP "*will watch this for commits and pull requests*".

Step 4: Configure settings and create (shots 08–09)

- **Workspace Name**: `demorepo` (unique within the org; dashes, underscores, alphanumerics)
- Description and Tags are optional. **Advanced options** hold the Terraform working directory (for a sub-folder in the repo), the VCS branch, auto-apply, and trigger path filters.
- Click **Create**. You get "*Workspace created! Next step: Configure Terraform variables*". Our code had no `variable` blocks, so it said **"No variables found"**.

Step 5: Workspace overview (shot 10)

Field	Value in the lab	Meaning
ID	`ws-ntpPVcq1AyXTfZaY`	Workspace ID (used by API / CLI)
Lock	**Unlocked**	Manual workspace lock. While locked, no runs can apply
Resources	0	Resources in this workspace's state
Terraform	**v1.15.7**	The version HCP will run (set per workspace)
Status	**Configuration uploaded successfully**	HCP pulled the code from GitHub
Repo	`srm71118/demorepo`	Connected VCS repo
Execution mode	**Remote**	Plan/apply runs on HashiCorp's workers, not your machine
Auto-apply API, UI & VCS runs	**Off**	A human must click **Confirm & apply** after every plan
Auto-apply run triggers	Off	Same, for runs triggered by another workspace
Auto-destroy	Off	Can schedule a destroy (good for sandboxes)
Project	Default Project	

Workspace left menu: **Overview · Runs · States · Search & import · Variables · Settings**.

Step 6: Give HCP the AWS credentials (shots 11–13)

The run happens on HashiCorp's machine, which **doesn't have your** `aws configure`. So the AWS keys must be stored in the workspace: **Variables → Add variable**.

Two variable categories:

Category	What it is	Example
Terraform variable	Must **match a** `variable` **block** in your code. Same as a `.tfvars` value. Tick **HCL** for lists/maps or interpolation	`instance_type = "t2.micro"`
Environment variable	Exported in the **Terraform runtime environment** before the run, like `export` on Linux	`AWS_ACCESS_KEY_ID`, `TF_LOG`, `TF_VAR_region`

We added two **Environment variables**, both ticked **Sensitive**:

- `AWS_ACCESS_KEY_ID`
- `AWS_SECRET_ACCESS_KEY`

Result: **Workspace variables (2)**, Value **"Sensitive - write only"**, Category **env**. Once it's saved, nobody (not even you) can read a sensitive value back in the UI or API. You can only overwrite it.

> The provider block has only `region`. The AWS provider automatically picks the keys up from those environment variables, exactly like `export AWS_ACCESS_KEY_ID=...` on your own server.

Also on this page:

- **Quick setup AWS dynamic credentials / Dynamic provider credential sets (New)**: the better, production way. HCP gets **short-lived credentials per run through OIDC** (an IAM role trusts HCP Terraform), so there are **no static access keys stored anywhere**.
- **Variable sets**: one set of variables (e.g. AWS creds) **shared across many workspaces**. Workspace variables override a variable set with the same key and category.

Step 7: First run, triggered from the UI (shots 14–20)

Runs → **New run** (or "Start new plan"):

- **Run name**: defaults to "*Triggered via UI*"
- **Run type**: **Plan and apply (standard)**. Other options are plan-only, refresh-only, and destroy.
- **Additional planning options**: targets, replace, and so on.

The plan looks exactly like CLI output: `+ instance_type: "t2.micro"`, `+ key_name: "ram_aws"`, and lots of *Known after apply*. Because **auto-apply is Off**, the run stops at **Apply pending** with a **Confirm & apply** box, where you must **add a comment explaining the action**. We wrote "*ok go with this plan...*". This comment is stored with the run, which gives you an audit trail of who approved what and why.

Code in the repo (`ec2.tf` in `srm71118/demorepo`, branch `main`):

```
provider "aws" {
  region = "ap-south-1"
}
resource "aws_instance" "linux" {
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  key_name      = "ram_aws"
  tags = {
    Name   = "Test_server"
    source = "terraform"
  }
}
```

Result in AWS (Mumbai): `Test_server`, `i-0ee11545846aabf1e`, **Running**, `t2.micro`, `ap-south-1b`, public IP `13.235.78.6`, private IP `172.31.14.110`.

Runs tab: *Current Run*: **Triggered via UI**, **Applied**, `tcaadmin triggered via UI`, Branch **main**, commit **83d4441**. The Run List has filters: **All · Needs Attention · Errored · Running · On Hold · Success**.

States tab: one state version, `#sv-kqd1egDe7cKjzRU1`, linked to run `#run-i6d97D9oFPgbQQfV` and commit `83d4441`. Resources = **1**. **The state file never touched our laptop or the repo.** It's stored, versioned and locked by HCP. There's also a **Lock** button and **+ New run** here.

Step 8: Notifications (shots 21–22)

Workspace Settings → Notifications → Create a Notification

Workspace settings menu: **General · Locking · Notifications · Policies · Run Tasks · Run triggers · SSH Key · Version Control · Destruction and deletion**.

Destination	Sends to
Webhook	POST messages to any URL
Email	Users of the org, via email. **We used this**
Slack	A Slack channel (incoming webhook)
Microsoft Teams	A Teams channel

Lab: Name `demo`, recipient = the org member's email, **Run Events = All events**. "All events" covers **Created, Planning, Needs attention, Applying, Completed, Errored**. The other choices are **No events** or **Only certain events** (in real life, usually just *Needs attention* + *Errored*).

Step 9: Enable cost estimation (shot 23)

This is an **Organization** setting, not a workspace one: **Organization Settings** → **Integrations** → **Cost estimation** → **"Enable cost estimation for all workspaces"** → Update settings. "*When possible, display an estimated monthly cost for resources provisioned.*"

Org settings menu: **General · Plan & billing · Tags · Teams · Users · Variable sets · Runs**, then under Integrations, **Cost estimation**.

Step 10: Change the code in GitHub and let HCP run on its own (shots 19, 24–28)

This is the real power of the **VCS workflow**. We **didn't open a terminal at all**:

1. Edited `ec2.tf` **directly in GitHub**: `instance_type = "t2.micro"` → `"t2.nano"`, and committed to `main` (commit message "Update ec2.tf").
2. GitHub's webhook told HCP → a new run **"Update ec2.tf"** started automatically: **"srm71118 triggered a run from GitHub"**. Plan running → *Refreshing aws_instance.linux* → **Cost estimation pending** → **Apply pending**.
3. The plan shows an **in-place update (`~`)**, not a replace:

```
~ aws_instance.linux
    id            : "i-0ee11545846aabf1e"
  ~ instance_type : "t2.micro" -> "t2.nano"
  ~ public_dns    : "ec2-13-235-78-6.ap-south-1.compute.amazonaws.com" -> (known after a
  ~ public_ip     : "13.235.78.6" -> (known after apply)
    37 unchanged attributes hidden · 9 unchanged blocks hidden
```

   Changing the instance type needs a **stop** → **modify** → **start**, so the instance keeps its ID but gets a **new public IP / DNS** (no Elastic IP). That's why those show *known after apply*.

4. **Cost estimation finished:** `1 of 1 estimated · $5.12/month · -$4.32`

Resource	Name	Hourly cost	Monthly cost	Monthly delta
aws_instance	linux	$0.007	$5.115	**-$4.315**

   So the t2.micro was about $9.43/month, and the change **saves $4.32/month**. You see the money impact **before** you click apply.

5. **Email notifications** arrived in Gmail from *HCP Terraform*, one per event:
   - `… helloyello123321456/demorepo - is in state Run Created` (08:45)
   - `… - is in state Run Planning` (08:45)
   - `… - Cost Estimated - Needs Confirmation - Action Required` (auto-apply is off, so it waits for us)

- `… - is in state Run Applying` (08:47, after we confirmed)

6. **States** tab now has **two versions**:
   - `Update ec2.tf`, `#sv-8SchwPKCa877UDpy`, *srm71118 triggered from Terraform*, run `#run-XypztTKomWi6aRBW`, commit `5297b1e`
   - `Triggered via UI`, `#sv-kqd1egDe7cKjzRU1`, *tcaadmin*, 7 minutes ago, commit `83d4441`

   Every state version maps to **who** ran it, **which run**, and **which Git commit**. That's full traceability, and you can roll back to or download any earlier state version.

---

3. The run lifecycle (what each status means)

```
Pending → Planning → Planned → Cost estimating → Policy check (Sentinel/OPA)
        → Needs confirmation ──(Confirm & apply + comment)──→ Applying → Applied
                      └──(Discard)──→ Discarded          (failure at any step → Errored)
```

- With **auto-apply On**, *Needs confirmation* is skipped (fine for dev, risky for prod).
- **Pull requests** get a **speculative plan** (plan only, can never apply), and the result is posted back on the PR as a status check. Reviewers see the plan **before** merging.
- **Merge to the tracked branch** (`main`) → real run → apply (after confirmation).
- Only **one run applies at a time per workspace**. Others queue, because the state is locked automatically.

---

4. CLI-driven workflow *(not done in the lab, but you should know it)*

Keep using your terminal while state and execution move to HCP. Add a `cloud` block:

```
terraform {
  cloud {
    organization = "helloyello123321456"
    workspaces {
      name = "demorepo-cli"
    }
  }
}
```

```
terraform login        # opens browser → creates an API token → saved in ~/.terraform.d/cred
terraform init         # connects the directory to the HCP workspace (offers to migrate exis
terraform plan         # runs REMOTELY; output streams to your terminal + link to the run in
terraform apply        # same; confirmation happens in the terminal or UI
```

- The `cloud` block **replaces the** `backend` **block** from Ch 08 (you can't have both).
- **Execution mode** per workspace: **Remote** (HCP runs it, the default), **Local** (your machine runs it and HCP only stores state, so it acts as a pure remote backend), **Agent** (runs on a self-hosted agent inside your private network, for private resources).
- A VCS-connected workspace **rejects** `terraform apply` **from the CLI**. Changes must go through Git. CLI `plan` is still allowed (speculative).

---

5. Terraform CLI + S3 backend vs HCP Terraform

Concern	CLI + S3/DynamoDB (Ch 08)	HCP Terraform
State storage	You build and secure the S3 bucket + versioning	Built-in, versioned, encrypted
Locking	DynamoDB table / `use_lockfile`	Automatic
Where it runs	Laptop / Jenkins agent	HashiCorp's isolated workers (or your agents)
Credentials	Every engineer has AWS keys	Stored once in the workspace (or OIDC dynamic creds)
Approval / audit	Whatever your CI provides	Confirm & apply + comment, run history per commit
Policy	None built in	Sentinel / OPA (paid)
Cost visibility	None	Cost estimation per run
Notifications	DIY	Email / Slack / Teams / Webhook
Price	AWS cost only	Free tier limited by managed-resource count, paid tiers per resource

When to choose what: small team or full control → CLI + S3 backend + your own CI pipeline. Many teams and environments, or a need for governance, approvals and audit → HCP Terraform (or Terraform Enterprise if it must be self-hosted). Open-source alternatives in the same space: Atlantis, Spacelift, env0, Scalr.

---

6. Gotchas and best practices

- **No AWS creds in the workspace → plan fails** with "*No valid credential sources found*". The remote worker doesn't have your `~/.aws/credentials`.
- **Always tick Sensitive** for secrets. Prefer **dynamic credentials (OIDC)** over static `AWS_ACCESS_KEY_ID` in production. Never commit keys to the repo HCP watches.
- **Keep auto-apply Off for prod**, so someone reviews the plan and confirms with a comment. You can allow auto-apply on dev/sandbox.
- **Repo sub-folders:** if the `.tf` files aren't at the repo root, set the **Terraform Working Directory** (Advanced options), or HCP plans an empty config.
- **Trigger path filters:** in a mono-repo, only run the workspace when its own folder changes.
- **Pin the Terraform version** in workspace settings to match `required_version` in the code.
- **Destroy:** Settings → *Destruction and deletion* → **Queue destroy plan** (deletes resources). Deleting the workspace alone **does not** delete AWS resources, it just orphans them.
- **Clean up after the lab:** queue a destroy plan so `Test_server` doesn't keep billing.

Command reference (this chapter)

```
# CLI-driven workflow
terraform login                          # get + store an HCP API token
terraform logout
terraform init                           # with a `cloud {}` block → link to HCP workspace
terraform plan                           # remote plan, streamed locally
terraform apply                          # remote apply (blocked if workspace is VCS-driven)
terraform init -migrate-state            # move existing local/S3 state into HCP

# Environment variables HCP (or your CI) commonly sets
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY   # static creds (Sensitive, env category)
TFC_AWS_PROVIDER_AUTH=true                  # dynamic (OIDC) credentials …
TFC_AWS_RUN_ROLE_ARN=arn:aws:iam::<acct>:role/<role>   # … the IAM role HCP assumes
TF_LOG=DEBUG                                # debug logs in the run output
TF_VAR_<name>=<value>                       # alternative way to feed Terraform variables
```

UI paths to remember:

Task	Path
New workspace	Workspaces → New Workspace → Version Control Workflow
Credentials	Workspace → Variables → Add variable → Environment variable → Sensitive
Manual run	Workspace → Runs → New run → Plan and apply (standard)
Approve	Run page → Confirm & apply → comment
Notifications	Workspace → Settings → Notifications → Create a Notification
Cost estimation	Organization Settings → Integrations → Cost estimation
State history	Workspace → States
Destroy everything	Workspace → Settings → Destruction and deletion → Queue destroy plan

---

Interview questions

1. **What is Terraform Cloud and how is it different from the Terraform CLI?** It's managed SaaS that runs the same Terraform remotely, with built-in state, locking, VCS triggers, RBAC, policies, cost estimation and run history.
2. **Organization vs Project vs Workspace?** Company → folder of workspaces → one config + its variables + state + runs.
3. **HCP workspace vs `terraform workspace`?** The CLI version is just multiple state files for one directory. An HCP workspace is a full unit with its own code source, variables, permissions and history.
4. **The 3 workflows?** VCS-driven (commits/PRs), CLI-driven (`cloud` block + `terraform login`), API-driven (pipelines).
5. **How does HCP get cloud credentials?** Workspace env variables marked Sensitive, a variable set shared across workspaces, or (best) dynamic provider credentials via OIDC.
6. **What is Sentinel?** HashiCorp's policy-as-code framework that checks every plan (security, cost, config rules) before apply. Policies can be advisory, soft-mandatory or hard-mandatory.
7. **What happens on a PR vs a merge?** PR → speculative plan posted to the PR. Merge → real run → confirm → apply.
8. **Terraform Enterprise?** The self-hosted version of the same platform.

Terraform course: complete 🎉

#	Chapter	Core idea
01	Workflow	Write → init → plan → apply → destroy
02	State & references	`terraform.tfstate` = desired vs current; implicit dependencies
03	Advanced commands	`-replace`, `import`, CLI workspaces, refresh, `TF_LOG`, user data
04	Provisioners	`file`, `remote-exec`, `local-exec` + `connection`
05	Plugin cache	`~/.terraformrc` `plugin_cache_dir`
06	Variables	Types, tfvars, precedence, outputs, locals
07	Data sources	Read-only lookups (existing SG, latest AMI)
08	Backend	Remote state in S3 + locking, migration
09	Modules	Root vs child, local/GitHub/Registry sources
10	Meta-arguments	count, for_each, depends_on, provider, lifecycle, versions, state rm/mv
11	**Terraform Cloud**	Org → workspace, VCS-driven runs, variables, notifications, cost estimation

Next steps to keep it sharp: HashiCorp **Terraform Associate** certification (the course this lecture series prepares for) · build one real project end to end (VPC + EKS module + remote state + CI pipeline) · try the CLI-driven workflow and Sentinel policies on HCP Terraform.